UniGe | MaLGa

MLCC 2026

Optimization for ML: SGD and friends

Cesare Molinari

MaLGa, UniGe-Dima

About this class

- What is optimization and why do we need it for ML?
- Gradient algorithms (SGD, the workhorse)

What is optimization?

Given $F : \mathbb{R}^d \to \mathbb{R}$ **smooth** (for now...) and **convex** (for now...),

$$\min_{w \in \mathbb{R}^d} \quad F(w)$$

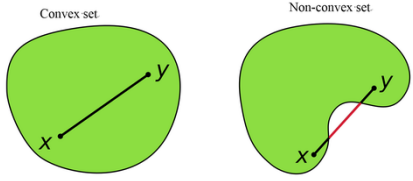
Indeed... find

$$w^* \in \arg\min_{w \in \mathbb{R}^d} \quad F(w)$$

- **Smoothness**: regularity, gradient available
- **Convexity**: see next...

What is convexity?

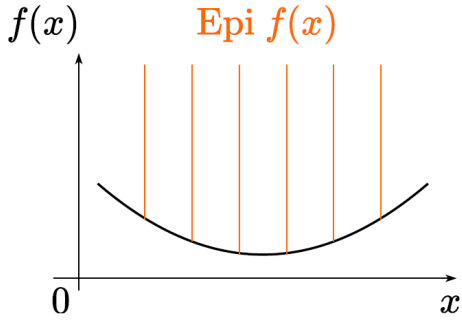
A geometrical property for sets (with topological implications):



- infinite generalization of polytopes (linearity)

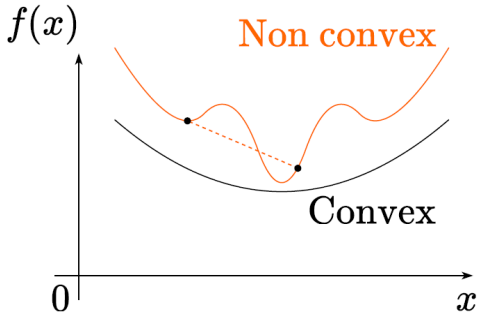
What is a convex function?

Epigraph: subset of $\mathbb{R}^d \times \mathbb{R}$



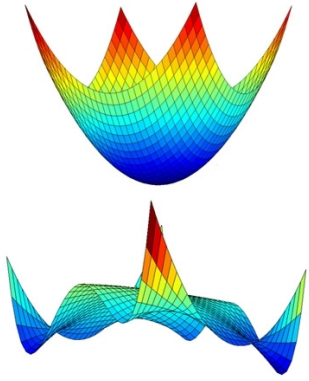
What is a convex function?

Convex function: the epigraph is a convex set!



What is a convex function?

Convex epigraph VS Non-Convex!



Why convexity?

Because it's easy: given $F : \mathbb{R}^d \to \mathbb{R}$ **smooth** and **convex**,

$$w^* \in \arg\min_{w \in \mathbb{R}^d} F(w) \quad \iff \quad \nabla F(w^*) = 0$$

- critical means min
- local means global

Then... How do we optimize?

Find

$$w^* \in \arg\min_{w \in \mathbb{R}^d} F(w)$$

Optimality condition: solve

$$\nabla F(w^*) = 0$$

BUT high-dimensional non-linear equations...

An abstract procedure

Gradient flow: for $t \geq 0$,

$$\begin{cases} \dot{w}(t) &= -\nabla F(w(t)) \\ w(0) &= w_0 \end{cases}$$

Main properties

- $\frac{d}{dt} F(w(t)) = \langle \nabla F(w(t)), \dot{w}(t) \rangle = - \|\dot{w}(t)\|^2 \leq 0$
- $\frac{d}{dt} \frac{1}{2} \|w(t) - w^*\|^2 = \langle \dot{w}(t), w(t) - w^* \rangle = -\langle \nabla F(w(t)), w(t) - w^* \rangle \underbrace{\leq}_{\text{convex}} - [F(w(t)) - F(w^*)] \leq 0$

How to obtain an algorithm?

Discretization in time! (Euler forward, explicit)

Gradient Descent:

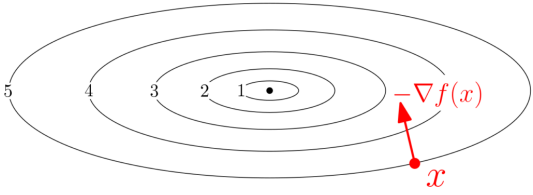
$$\begin{cases} \frac{w_{k+1}-w_k}{\gamma} & = -\nabla F(w_k) \\ w_0 \end{cases}$$

that means

$$w_{k+1} = w_k - \gamma \nabla F(w_k)$$

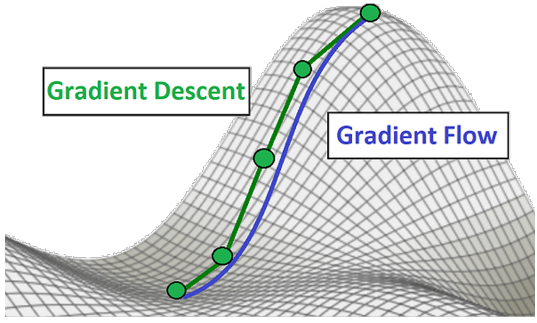
- 1st-order method - it uses only first derivatives...

Gradient Descent



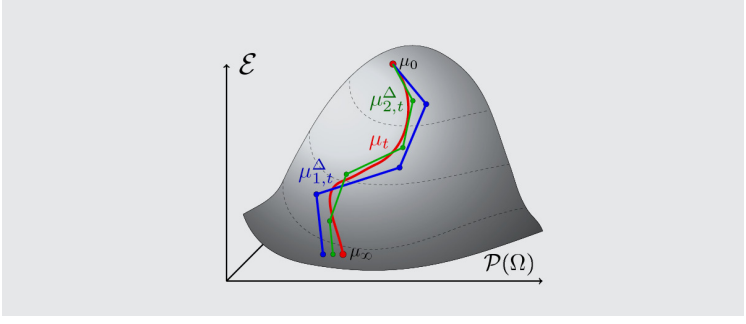
Flow VS Descent

Continuos VS Discrete



Flow VS Descent

Continuos VS Discrete



Theoretical guarantees

Choosing $\gamma < 2/L$, for every $w_0 \in \mathbb{R}^d$

- **Horizontally:**
$$w_k \to w_\infty \in \arg\min F$$

- **Vertically:** $F(w_k) \searrow$ with
$$F(w_k) - F(w^*) \leq \frac{C}{k}$$

A different range of results can be obtained changing assumptions, e.g. F strongly convex

The step-size matters!

For $F(w) = \frac{1}{2}\|w\|^2$, $\nabla F(w) = w$; GD reads

$$w_{k+1} = w_k - \gamma \nabla F(w_k) = (1-\gamma) w_k$$

and so

$$w_k = (1-\gamma)^k w_0$$

- If $\gamma = 0.9$ or $\gamma = 1.9$,

$$w_k \ \to \ 0 \quad \text{(exponentially fast)}$$

- If $\gamma = 2.1$,

$$\|w_k\| \ \to \ +\infty$$

A different interpretation

GD: $w_{k+1} = w_k - \gamma \nabla F(w_k)$ is equivalent to

$$w_{k+1} \in \arg\min_{w} \quad \underbrace{F(w_k) + \langle \nabla F(w_k), w - w_k \rangle + \frac{1}{2\gamma} \|w - w_k\|^2}_{\text{linear}}$$

Can we do better?

Newton method

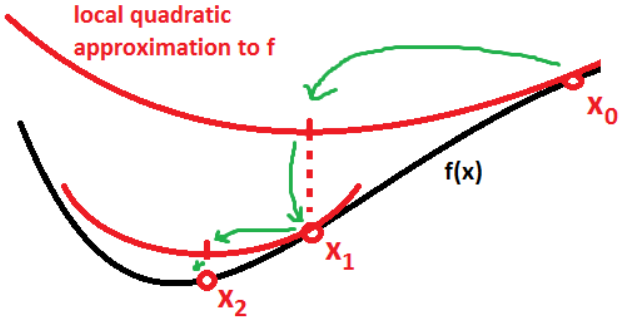
Given $F : \mathbb{R}^d \to \mathbb{R}$ smooth (twice differentiable) and convex,

$$w_{k+1} = w_k - \left[\nabla^2 F(w_k)\right]^{-1} \nabla F(w_k)$$

Why?

$$w_{k+1} \in \arg\min_w \quad \underbrace{F(w_k) + \langle \nabla F(w_k), w - w_k \rangle + \frac{1}{2}\langle \nabla^2 F(w_k)\,(w - w_k)\,, w - w_k \rangle}_{\text{quadratic}}$$

Newton



Newton method

Steps in minus the gradient + considering curvature of the function

BUT: 2nd-order method... computing and inverting the Hessian at each step is not feasible/efficient!

From Newton back to GD

Observation: in 1d, Newton is just an **adaptive** step-size choice

$$w_{k+1} = w_k - \gamma_k F'(w_k), \quad \text{with } \gamma_k = [F''(w_k)]^{-1}$$

So... back to Gradient Descent, revisited:

$$w_{k+1} = w_k - \gamma_k \nabla F(w_k), \quad \text{with } (\gamma_k)_k \text{ “suitable” step-size sequence}$$

$\longrightarrow$ 1st-order method but with “variable metric”

And even more to come to alleviate computations in ML case! (SGD)

Back to ML

ERM? it's an optimization problem:

$$\min_{w \in \mathbb{R}^d} \hat{\mathcal{E}}_\lambda(w) := \hat{\mathcal{E}}(w) + \lambda \|w\|^2$$

with

$$\hat{\mathcal{E}}(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, w^\top x_i)$$

Remark: if $\ell(y, \cdot)$ is convex (ex: least squares, logistic), then...

$\hat{\mathcal{E}}_\lambda$ is convex!

Least Squares

w^* is a minimizer iff

$$\nabla \hat{\mathcal{E}}_\lambda(w^*) = 0$$

Least squares: $\ell(y, w^\top x) = (y - w^\top x)^2$, quadratic error + quadratic norm

$$\hat{\mathcal{E}}_\lambda(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

and so

$$\nabla \hat{\mathcal{E}}_\lambda(w) = (X^\top X + \lambda I)w - X^\top y$$

$\longrightarrow d \times d$ linear system!

Logistic

w^* is a minimizer iff

$$\nabla \hat{\mathcal{E}}_\lambda(w^*) = 0$$

Logistic regression: $\ell(y, w^\top x) = \log(1 + e^{-y w^\top x})$,

$$\hat{\mathcal{E}}_\lambda(w) = \frac{1}{n} \sum_{i=1}^{n} \log(1 + e^{-y_i w^\top x_i}) + \frac{\lambda}{2} \|w\|^2$$

and so

$$\nabla \hat{\mathcal{E}}_\lambda(w) = -\frac{1}{n} \sum_{i=1}^{n} \frac{y_i}{e^{y_i x_i^\top w} + 1} x_i + \lambda w$$

$\longrightarrow d \times d$ non-linear system of equations...

Solving ERM

Then... back to optimization:

$$\min_{w \in \mathbb{R}^d} \hat{\mathcal{E}}_\lambda(w)$$

How do we solve it?

(Spoiler: SGD)

GD for Logistic Regression

Recalling

$$\nabla \hat{\mathcal{E}}_\lambda(w) = -\frac{1}{n}\sum_{i=1}^{n} \frac{y_i}{1+e^{y_i x_i^T w}} x_i + 2\lambda w,$$

the GD iteration becomes

$$w_{k+1} = w_k - \gamma \left(-\frac{1}{n}\sum_{i=1}^{n} \frac{y_i}{1+e^{y_i x_i^T w_k}} x_i + 2\lambda w_k \right)$$

GD: the goods and the bads

$$w_{k+1} = w_k - \gamma \left(-\frac{1}{n} \sum_{i=1}^{n} \frac{y_i}{1 + e^{y_i x_i^T w_k}} x_i + 2\lambda w_k \right)$$

- requires only vector multiplications, hence easily parallelizable
- possibly slow and at each iteration all the data needs be stored and processed

From GD to SGD

For a general loss, **GD** is

$$w_{k+1} = w_k - \gamma \left(\frac{1}{n} \sum_{i=1}^{n} \nabla_w \ell(y_i, x_i^\top w) \Big|_{w_k} + 2\lambda w_k \right)$$

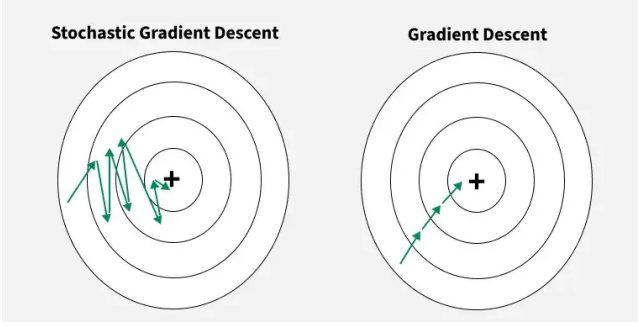
while **SGD** corresponds to

$$w_{k+1} = w_k - \gamma_k \left(\nabla_w \ell(y_{i_k}, x_{i_k}^\top w) \Big|_{w_k} + 2\lambda w_k \right)$$

where i_k is a random variable on $[n]$, e.g. uniform

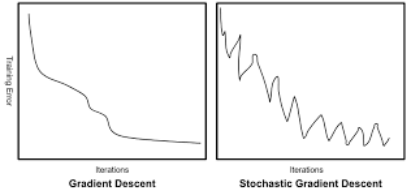
- Only one data (x_i, y_i) is processed at each iteration!

SGD vs GD



More “chaotic”

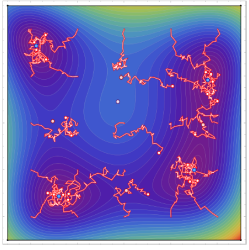
SGD vs GD



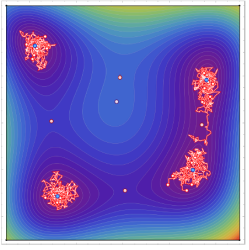
More “chaotic”:

- not a descent method ($F(w_k)$ not decreasing)

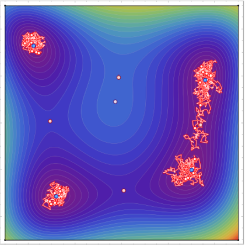
SGD



$t = 50$



$t = 200$



$t = 500$

More “chaotic”:

- convergence to error-dominated-region

Why SGD?

Small time/memory cost per iteration

Possibility of dealing with streaming data:

$$w_{k+1} = w_k - \gamma_k \left(\nabla \ell(y_k, x_k^\top w_k) + 2\lambda w_k \right)$$

Convergence guarantees

Assumptions:

- L-**smoothness** (∇F is Lipschitz continuous)
- **Convexity** (of F)
- **Stochastic Oracle**: unbiased with bounded variance

$$\mathbb{E}[g_k \mid w_k] = \nabla F(w_k), \quad \mathbb{E}[\|g_k - \nabla F(w_k)\|^2 \mid w_k] \leq \sigma^2$$

Constant step-size

If $\gamma_k = \gamma \leq \frac{1}{L}$,

$$\mathbb{E}[F(\bar{w}_k) - F(w^*)] \leq \frac{\|w_o - w^*\|^2}{2\gamma k} + \frac{\gamma \sigma^2}{2}$$

Optimal step-size (horizon known)

For K iterations, choosing $\gamma = \frac{R}{\sigma\sqrt{K}}$,

$$\mathbb{E}[F(\bar{w}_K) - F(w^*)] \leq \frac{R\sigma}{\sqrt{K}} = \mathcal{O}\left(\frac{1}{\sqrt{K}}\right)$$

SGD Flavors

SGD is hardly ever used in the basic form

$$w_{k+1} = w_k - \gamma_k \left(\nabla \ell(y_{i_k}, x_{i_k}^\top w_k) + 2\lambda w_k \right).$$

Possible variations:

- data selection
- mini-batching
- acceleration
- averaging

Data sampling

$$w_{k+1} = w_k - \gamma_k \left(\nabla \ell(y_{i_k}, x_{i_k}^\top w_k) + 2\lambda w_k \right).$$

- Sample data uniformly at random
- Visit data in a fixed prescribed order
- Visit data sequentially, reshuffling after each pass (epoch)

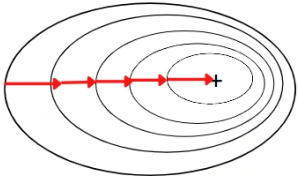
Mini-batching

$$w_{k+1} = w_k - \gamma_k \left(\frac{1}{b} \sum_{j \in B,\ |B|=b} \nabla \ell(y_j, x_j^\top w_k) + 2\lambda w_k \right)$$

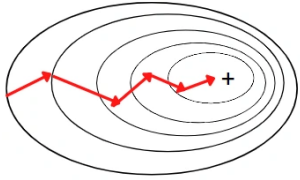
- Uses b points at each iteration for a more accurate gradient estimate
- Allows efficient memory use
- First step towards using distributed resources

Batching

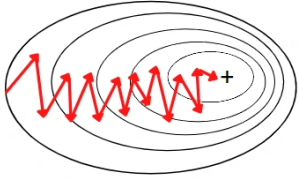
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Acceleration: momentum

$$\begin{aligned} v_k &= w_k + \beta_k(w_k - w_{k-1}) \\ w_{k+1} &= v_k - \gamma_k \left(\nabla \ell(y_{i_k}, x_{i_k}^\top v_k) + 2\lambda v_k \right) \end{aligned}$$

- The momentum/inertia is $w_k - w_{k-1}$ (Nesterov)
- Two previous iterations are used, with potential increased speed

(Polyak) averaging

$$\overline{w}_k = \frac{1}{k} \sum_{j=s}^{k} a_j w_j$$

- $s = 1,\ a_k = 1$ uniform averaging
- $s > 1$ tail averaging
- $a_k \neq 1$ weighted (Cesàro) averages

Further remarks

- SGD can be extended to non smooth but convex functions keeping all guarantees!

- SGD can be extended to non convex functions but... loosing most guarantees!

Summing up

- First order methods dominate modern ML

- SGD is the method of choice, but with additional tricks!

Next class

... beyond linear models (and beyond convexity)!